



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

22/03/2026

NOMBRE DEL ALUMNO

CARLOS EDUARDO HERNANDEZ CAMACHO

GRUPO

8^a

ACTIVIDAD

MANEJO DE MULTISESIONES Y PERSISTENCIA

1. ¿Qué es una multisesión?

Una multisesión ocurre cuando un mismo usuario tiene varias sesiones activas al mismo tiempo en un sistema. Esto puede pasar, por ejemplo:

- **Cuando un usuario inicia sesión en su cuenta desde varios dispositivos (PC, móvil, tablet).**
- **Cuando un usuario abre varias ventanas o pestañas en un navegador con la misma cuenta.**

Ejemplo práctico: Tu cuenta de correo está abierta en tu teléfono y en tu laptop simultáneamente; cada dispositivo tiene una sesión independiente.

2. ¿Qué problemas surgen con sesiones concurrentes?

Las sesiones concurrentes pueden generar varios problemas de seguridad y funcionalidad:

- 1. Riesgo de robo de sesión: Si un atacante obtiene la sesión de un dispositivo, podría acceder a tu cuenta incluso si tú estás usando otro dispositivo.**
- 2. Inconsistencias de datos: Acciones simultáneas desde diferentes sesiones pueden generar conflictos en bases de datos (por ejemplo, edición de un mismo registro).**
- 3. Difícil control de permisos: Si un administrador cambia los permisos de un usuario, las sesiones antiguas podrían seguir teniendo acceso hasta expirar.**
- 4. Consumo de recursos: Más sesiones activas significan más memoria y conexiones en el servidor, lo que puede afectar rendimiento.**

3. ¿Cómo se invalida una sesión?

Existen varias formas de invalidar o cerrar una sesión:

- 1. Expiración automática: Configurar que las sesiones expiren después de un tiempo de inactividad.**
- 2. Cierre manual: El usuario cierra sesión desde la aplicación (“Logout”), eliminando la cookie o token.**

3. **Revocación desde el servidor:** El administrador o el sistema puede invalidar la sesión eliminando el registro en la base de datos o marcando el token como inválido.
4. **Rotación de tokens:** En sistemas con JWT, cambiar el secret o invalidar el token antiguo fuerza al usuario a autenticarse nuevamente.

4. ¿Qué riesgos existen si no se controlan correctamente?

Si no se gestionan las sesiones correctamente, pueden surgir graves riesgos de seguridad:

- **Secuestro de sesión (Session Hijacking):** Un atacante puede usar una sesión activa sin necesidad de conocer la contraseña.
- **Acceso no autorizado:** Usuarios con sesiones antiguas o compartidas podrían acceder a recursos que ya no deberían.
- **Fuga de información:** Sesiones concurrentes no controladas pueden exponer datos sensibles en varios dispositivos.
- **Abuso de privilegios:** Usuarios con varias sesiones pueden evadir bloqueos o restricciones temporales.

Actividades DevOps / CI-CD (SGIM)

Se realizaron ajustes para preparar el proyecto para entornos de desarrollo y despliegue, asegurando seguridad, configuración adecuada y validación automática del código.

Variables de entorno

- Se creó y actualizó `.env.example` con variables para:
 - Base de datos (PostgreSQL)
 - JWT (JWT_SECRET, JWT_EXPIRES_IN)
 - Sesiones (SESSION_SECRET, SESSION_EXPIRES_IN)
- Se estandarizó la configuración para uso en distintos entornos (dev/staging/producción).

Seguridad

- Se aseguró que .env no se suba al repositorio mediante .gitignore.
- Se eliminó backend/.env del repo para evitar exposición de secretos.
- Se mantuvo .env.example como referencia segura.

Manejo de sesiones

- Se implementó manejo de sesión en frontend:
 - Persistencia con localStorage
 - Logout normal y logout global (multisesión)
 - Manejo de errores y estados de carga
- Se protegieron rutas por roles (usuario y admin).

Migraciones

- Se verificó que no existen migraciones para tabla sessions.
- Se documentó que la gestión actual de sesiones es a nivel frontend.
- Se recomendó implementación futura en backend para producción.

CI/CD

- Se configuró pipeline con GitHub Actions:
 - Ejecución automática en Pull Requests
 - Tests del backend (npm test)
 - Build del frontend (npm run build)
- Se garantiza validación automática antes de integrar cambios.

Conclusión

El proyecto quedó preparado con buenas prácticas de DevOps:

- Configuración segura de entorno
- Protección de datos sensibles
- Automatización de validaciones
- Base lista para despliegue y escalabilidad

Los cambios fueron realizados y sincronizados en la rama main.